

The C++ fast loop

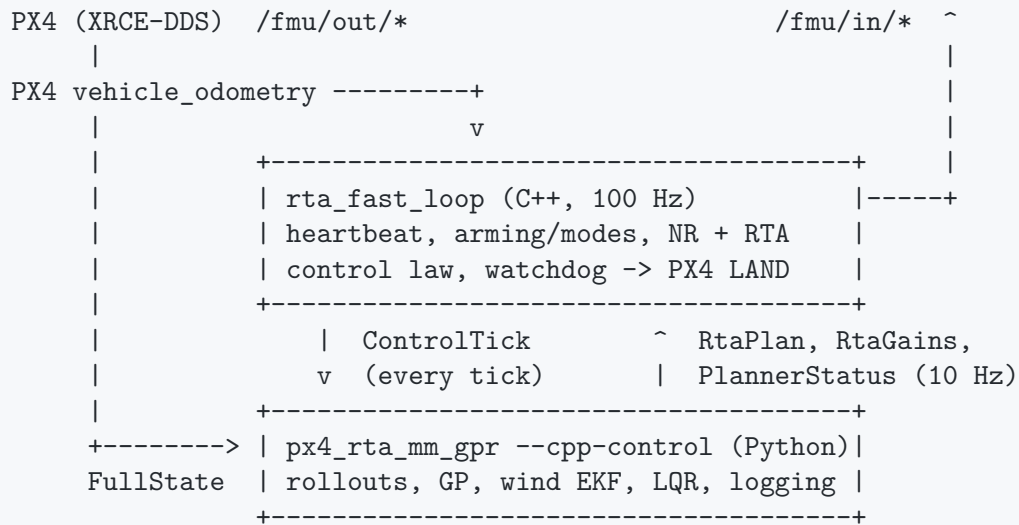
Branch `cpp-version` (= main + this layer): PX4 I/O, the 100 Hz control law and the runtime-assurance watchdog in C++; rollouts, GP and wind estimation stay in Python

2026-09-28

Table of contents

1	Architecture	1
2	The control law in C++	2
3	Threads in the C++ node	3
4	Results	3
5	Running it	5
6	Next steps	5

1 Architecture



	Python planner	C++ <code>rta_fast_loop</code>
owns	mission clock, all tunables, rollouts (AOT JAX, worker process), GP data, wind EKF, LQR gains, the HDF5 flight log	everything PX4 sees: heartbeat, offboard/arm, setpoints, the NR + RTA control law, the watchdog
publishes	<code>PlannerStatus</code> (10 Hz), <code>RtaPlan</code> (per plan), <code>RtaGains</code> (per update)	<code>ControlTick</code> (per control tick)
if the other side dies	nothing to fly; logging stops	the plan expires uncertified, and after the grace period the watchdog commands LAND

Single source of truth. Every tunable the control law needs (mass, thrust and roll-rate limits, anti-windup bound, NR parameters, mission times, backup policy) arrives in `PlannerStatus`. The fast loop has no duplicated configuration, and it does not start the mission before the planner is alive and compiled. That is the same “T0 after compilation” rule the single-process node had.

Packages. The repository is multi-package (on `main` too, because `colcon` never looks for packages inside another package’s directory); this branch adds the two C++-side packages:

```
PX4_RTA_MM_GPR/
  docs/
  packages/
    px4_rta_mm_gpr/          # the Python package (was the repository root)
    px4_rta_mm_gpr_msgs/    # PlannerStatus, RtaPlan, RtaGains, ControlTick
    px4_rta_mm_gpr_cpp/     # rta_fast_loop, control_law.hpp
    flight_recorder/        # git submodule: flight-data logging library
```

```
colcon build --symlink-install \
  --base-paths PX4_RTA_MM_GPR/packages <other deps>
ros2 launch px4_rta_mm_gpr cpp_rta_launch.py
```

2 The control law in C++

`include/px4_rta_mm_gpr_cpp/control_law.hpp` is ROS-free: dynamics, forward-Euler prediction, the NR step with its integral CBF, the RTA feedback, the fused step with anti-windup, the throttle maps, Euler angles and yaw unwrapping, each ported term by term from the Python/JAX code.

The NR Jacobian with dual numbers. The NR tracker inverts $\partial g / \partial u$, the 4×4 Jacobian of the predicted output (x, y, z, yaw after 0.8 s) with respect to the four inputs. JAX computes it with forward-mode autodiff. The C++ port uses a dual number with *four* tangent directions:

$$\tilde{u}_i = u_i + \varepsilon_i, \quad f(\tilde{u}) = f(u) + \sum_i \frac{\partial f}{\partial u_i} \varepsilon_i,$$

where the ε_i are formal symbols with $\varepsilon_i \varepsilon_j = 0$. Every arithmetic operation and every `sin/cos/tan` propagates the value and all four partial derivatives at once (`Jet<4>`, about 40 lines). One pass of the 8-step Euler predictor then yields the prediction *and* the exact Jacobian: no step size to tune, no truncation error, and no dependency beyond Eigen.

Equivalence with the Python kernels. `test/test_control_law.py` feeds 1000 random cases through both implementations (normal flight, wound-up NR channels, active CBF, saturated RTA feedback, large attitudes):

NR Jacobian condition number	max relative difference, C++ vs JAX
$< 10^3$ (about 80 % of cases; all normal flight)	$3\text{--}5 \times 10^{-13}$
$10^3 - 10$	2×10^{-1}
> 10 with anti-windup (default)	3×10^{-11}
> 10 without anti-windup (± 50 rad/s rates)	3×10

The only visible differences are rounding differences between XLA and Eigen, amplified where the tracker inverts a nearly singular Jacobian. That is the wound-up regime that anti-windup exists to prevent.

3 Threads in the C++ node

`rclcpp::executors::MultiThreadedExecutor` with two threads and two mutually exclusive call-back groups: all subscriptions in one, the control and heartbeat timers in the other. A 10 KB plan message being deserialised can therefore never delay a control tick. The state, plan, gains and configuration cross between them as immutable snapshots:

```
std::atomic<std::shared_ptr<const Plan>> plan_;           // C++20
plan_.store(std::make_shared<const Plan>(...));          // subscription
↪ thread: build, then publish
const auto plan = plan_.load();                          // control thread:
↪ one load per tick
```

This is the same pattern as the Python node (`self.plan = plan`), with the atomicity made explicit. There is one ordering subtlety: the values derived from the first `PlannerStatus` (limits, NR parameters) are written *before* the config pointer is published, so the control thread can never see a config without them.

4 Results

Same mission and settings as the round-4 Python-loop flights (`r5_tuned`: goal 1.0 m, floor 0.3 m, events executor + process rollouts for the planner). Five nominal flights each, plus two tests of the safety path. Full per-flight numbers: `docs/data/round5_flights.csv`.

	Python loop (round 4)	C++ fast loop + Python planner
completed the RTA phase	5 / 5	5 / 5
final altitude (goal 1.00 m)	1.06–1.12 m	1.08–1.09 m
lowest altitude / lowest certified tube	0.98–1.03 m / 0.85–0.89 m	0.95–1.02 m / 0.82–0.90 m
ticks without a certified plan	0 %	0 %
control period p99 / max	10.06–10.42 ms / 12.4 ms	10.04 ms / 11.4 ms
control-law computation, median / max	118–148 μ s / 0.8–3.3 ms	4.3 μs / 41 μs

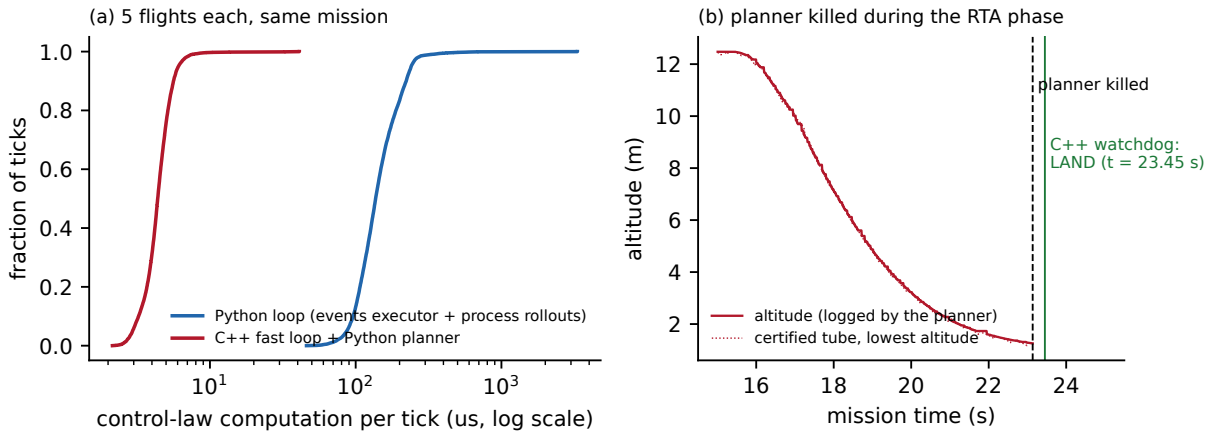


Figure 1: (a) Computation time of the control law per tick, all ticks of five flights each. (b) The Python planner was killed at mission time ~ 23.1 s, in the middle of the RTA phase. Its log ends there; the C++ fast loop kept flying the last certified plan and commanded LAND at 23.45 s, when that plan’s certified horizon ran out.

Safety tests.

- *Negative test* (floor 1.2 m, above the 1.0 m goal): certification failed at 1.20 m and the C++ watchdog commanded LAND at 23.82 s. The reported uncertified time is 30 ms against the 20 ms grace, because the check runs once per 10 ms tick.
- *Planner killed mid-flight*: the Python planner was killed during the RTA phase. The fast loop kept flying the last certified plan until its safety horizon ended, then commanded LAND by itself (at $t = 23.45$ s), and PX4 reported *Landing detected* and *Disarmed by landing*. In the single-process design the same failure would have taken the controller down with the planner. With the split, the watchdog doesn’t depend on the thing it watches.

What C++ changed, and what it didn’t. After rounds 2–4, the Python loop already held its 10 ms period on this 14-core desktop, so the flights look the same. The differences are margin and isolation:

1. about $30\times$ less computation per tick (4 μ s vs 130 μ s), and a tail bounded by tens of microseconds rather than milliseconds. That is the margin a slower onboard computer needs;

2. no GIL, garbage collector or JIT anywhere in the loop, by construction rather than by mitigation;
3. a watchdog that survives the planner.

5 Running it

```
PX4_RTA_MM_GPR/tools/run_sitl.sh          # PX4 SITL + x500 (sets
↪ NAV_DLL_ACT=0); then MicroXRCEAgent udp4 -p 8888
cd ~/<workspace>
colcon build --symlink-install --base-paths PX4_RTA_MM_GPR/packages deps
source install/setup.bash
ros2 launch px4_rta_mm_gpr cpp_rta_launch.py
# equivalence test of the C++ control law against the Python kernels
python3 PX4_RTA_MM_GPR/packages/px4_rta_mm_gpr_cpp/test/test_control_law.py
↪ \
    install/px4_rta_mm_gpr_cpp/lib/px4_rta_mm_gpr_cpp/control_law_check
```

All planner flags from documents 02–04 still apply (they reach the fast loop through `PlannerStatus`); without `--cpp-control`, the Python node flies by itself as before.

6 Next steps

- Hardware: the fast loop uses the hardware throttle map when `PlannerStatus.sim` is false. Test on the vehicle’s onboard computer with the planner in a separate process, or on a separate core (`--rollout-cpus`).
- The backup is PX4 LAND. A certified backup (for example, a hover plan certified in advance) would make the fallback itself part of the guarantee.